

2026 US LLVM Developers' Meeting

27 October 2026

How to build a Python DSL for MLIR

`mlir dsl` · Python generates MLIR

Guray Ozen

Principal Compiler Engineer, NVIDIA

Amir Tavakkoli

How do we produce MLIR?

Write IR by hand

Literal programs
Manual variants

Python bindings

Build operations
Wire compilation + ABI

Python DSL

Write the computation
Reuse the infrastructure

We want to write the computation

Same PyTorch CPU tensors · same $a + b$

Raw bindings · setup omitted

```
with ir.InsertionPoint(fn.add_entry_block()):
    n, x, y, z = fn.arguments
    lo, step = arith.ConstantOp(i32,0), arith.ConstantOp(i32,1)
    loop = scf.ForOp(lo, n, step)
    with ir.InsertionPoint(loop.body):
        i = loop.induction_variable
        addr = [llvm.GEPOp(ptr,p,[i],[DYN],f32,"None").result
                for p in (x,y,z)]
        lhs, rhs = [llvm.LoadOp(f32,p).result for p in addr[:2]]
        llvm.StoreOp(arith.AddFOp(lhs,rhs).result, addr[2])
        scf.YieldOp([])
    func.ReturnOp([])
PassManager.parse(PIPELINE).run(module.operation)
engine = ExecutionEngine(module)
args = [c_int32(a.numel())]
args += [c_void_p(t.data_ptr()) for t in (a,b,out)]
engine.invoke("add", *[byref(v) for v in args])
```

mlir dsl

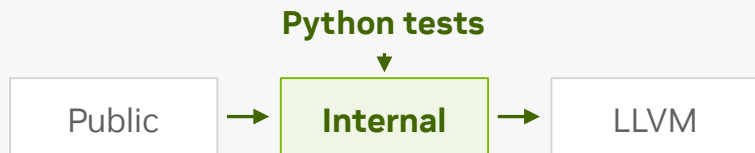
```
import mlir.mlir_dsl as m
P = m.Pointer[m.Float32]

@m.jit
def add(n: m.Int32, a: P,
        b: P, out: P):
    for i in range(n):
        out[i] = a[i] + b[i]

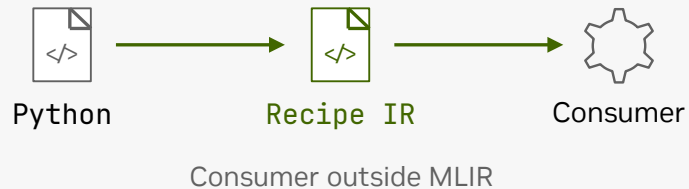
add(a.numel(), a, b, out)
# out: [10., 11., 12., 13.]
```

Four use cases inside NVIDIA

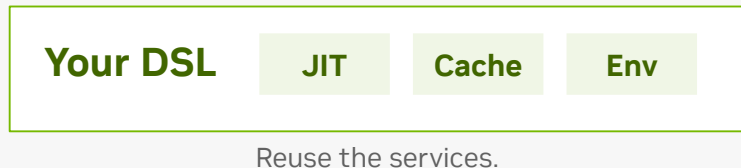
01 Test internal dialects



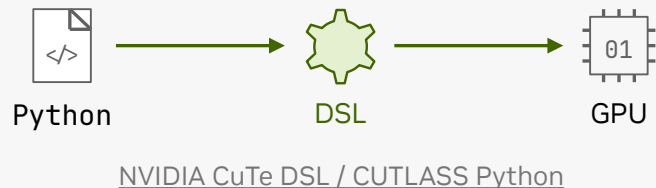
02 Generate recipe IR



03 Reuse DSL building blocks



04 Build a full language



MLIR has many worlds

Different dialects.

Different pipelines.

Different control flow.

One fixed language cannot serve every compiler.

Proposal: a dialect-agnostic core

Core DSL

Diagnostics

Errors · remarks

AST preprocessing

Plugin hooks

JIT cache

Memory · disk

Type inference

Promotion · casts

No default dialect. No default compiler.

Four roles. Two families.

Eight shipped plugins. Op modules are not plugins.

At most one per role

Type operations

Types + operators

TypeOps

Function entry

Entry + results

func.Entry

AST preprocessor

Syntax capture

scf.ASTPreprocessor

Compiler

Lower + invoke

ExecutionEngine compiler

Any number

Decorators

Add @kernel

gpu.Kernels

Adapters

Inputs + exports

PyTorch · DLPack · TVM-FFI

Shared core

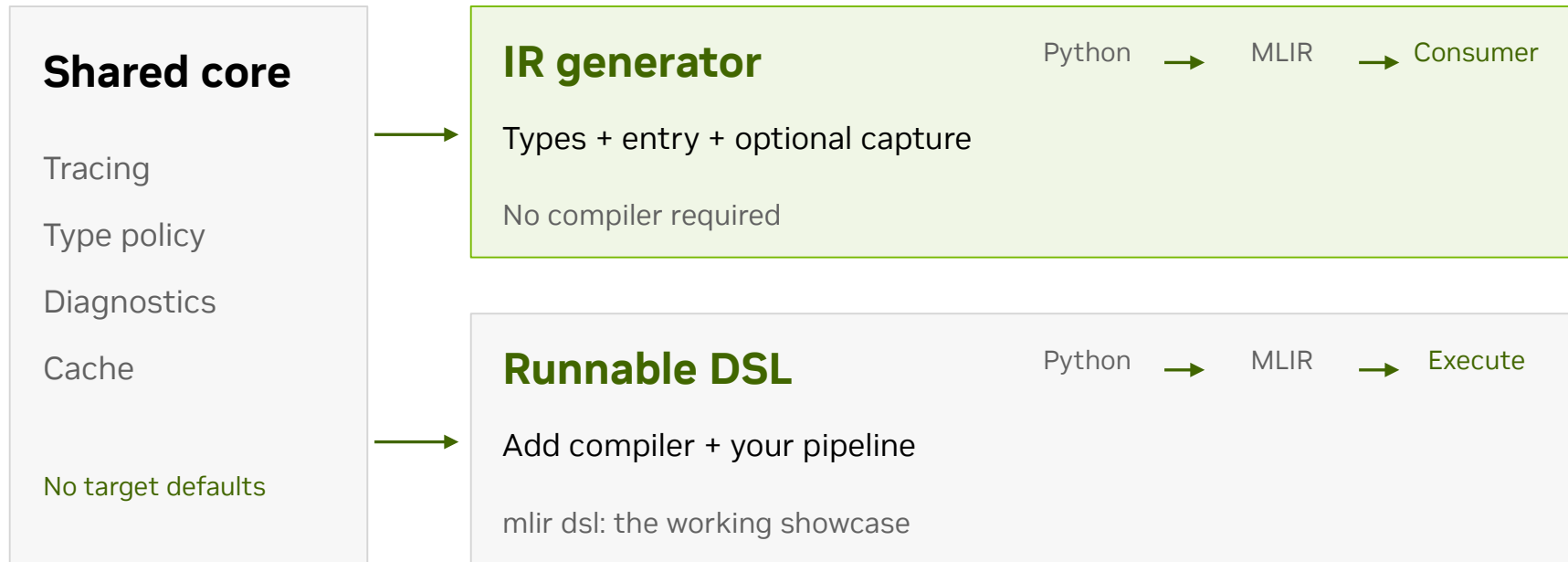
Tracing

Types

Diagnostics

Cache

Generate IR—or build an end-to-end DSL



Your DSL can reuse the same plugins.

Why tracing? Keep Python in Python.

AST-based frontend

Python AST → Analyze + lower → Target IR

Own the language semantics

Objects · calls · control flow

Tracing core

Run Python → DSL operations → Target IR

Python executes the metaprogram

Plugins choose the emitted IR

Small AST capture remains optional for staged control flow.

Design principles

01 No new language.

Objects · imports · libraries

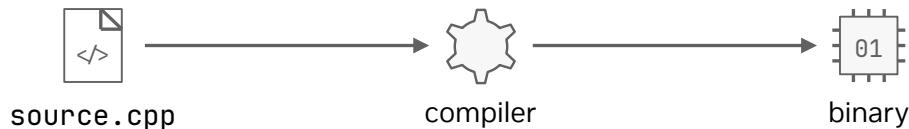
02 Python stays Python.

MLIR through a DSL namespace

This leads to **multi-stage programming**.

What is multi-stage programming?

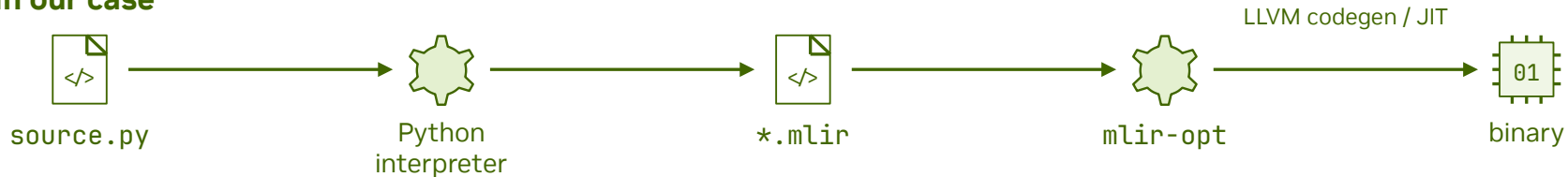
Traditional compiler



Multi-stage compilers



In our case



Why multi-stage programming?

01 METAPROGRAM

All of Python.

Objects · imports · libraries

02 DSL NAMESPACE

Explicit capture.

Types · operators · builders

```
import mlir.mlir_dsl as m
```

```
range(4)
```



Python loop

Runs now

```
m.range(4)
```



scf.for

Runs later

Agenda

- 01 Trace Python → types → control flow
- 02 Host boundary → kernels → compilation
- 03 Tools → build your own DSL
- 04 Amir: advanced mutation
- 05 Tile IR: a target composition

1 · Tracing JIT

User program

```
import mlir.mlir_dsl as m

@m.jit
def scale(x: m.Int32, factor):
    return x * factor

int(scale(4, 3))  # 12
```

Reference execution path

```
Python call
  ↓
Trace func.func
  ↓
Compile + cache
  ↓
Invoke
```

2 · Python value or MLIR value?

Python

```
@m.jit
def scale(x: m.Int32, factor):
    assert m.is_mlir_op(x)
    assert not m.is_mlir_op(factor)
    return x * factor

scale(4, 3)
```

Generated MLIR · excerpt

```
# At trace time
factor = 3

# In the generated program
%x : i32
%c3 = arith.constant 3 : i32
%r = arith.muli %x, %c3 : i32
```

Python is the metaprogram

Python · implementation excerpt

```
@m.jit
def affine(x: m.Int32, options):
    scale = options["scale"]
    bias = options["bias"]
    return x * scale + bias

options = {"scale": 3, "bias": 4}
affine(7, options) # Int32(25)
```

Runs in Python

Dictionary lookups

Captured as MLIR

Multiply + add

Objects + dictionaries → arithmetic

Python polymorphism

```
class Op:
    def apply(self, x):
        raise NotImplementedError

class AddN(Op):
    def __init__(self, n): self.n = n
    def apply(self, x): return x + self.n

class MulN(Op):
    def __init__(self, k): self.k = k
    def apply(self, x): return x * self.k
```

Metaprogram

```
@m.jit
def pipeline(x: m.Int32, ops):
    for op in ops.values():
        x = op.apply(x)
    return x

ops = {"shift": AddN(2), "scale": MulN(3)}
pipeline(4, ops) # 18

# Captured: (x + 2) * 3
```

No runtime dictionary or virtual dispatch.

3 · Numeric types are core policy

Family	Types
Integers	Int2–Int128 · Uint8–Uint128
Floating point	Float16/32/64 · BFloat16 · TFloat32
Narrow floats	FP8 · FP6 · FP4 formats
Boolean	Boolean

Literal → promotion → cast

Boolean · i1

Int2-128 / UInt8-128

FP64/32/16 · BF16

FP8/6/4 · TF32

Python or SSA value → dtype + promotion → type_ops plugin

Conversion

Python

MLIR

Capture

m.Int32(7) → staged use

arith.constant : i32

Promote / upcast

x32 + y64

extsi → addi : i64

Downcast

y64.to(m.Int32)

arith.trunci

Int → float

x32.to(m.Float32)

arith.sitofp

Float → int

f32.to(m.Int32)

arith.fptosi

Error

x32.to(int)

Runtime ≠ Python value

Shown: TypeOps with arith

The same dtype can target another dialect

Type-ops plugin · implementation excerpt

```
class TileTypes(TypeOps):  
    def mlir_type(self, dtype):  
        return ir.RankedTensorType.get(  
            [], dtype.scalar_mlir_type)  
  
class TileLike(m.MlirTestDSL):  
    plugins = replace(  
        m.MlirTestDSL.plugins,  
        type_ops=TileTypes(scalars=arith))
```

Same source

m.Int32(6) + x

Different IR type

i32 → tensor<i32>

4 · Vectors, pointers, structs

Value	How it maps
Vector	vector hooks: create · extract · reduce
Pointer[T]	memory hooks: load · store · offset
@struct	Frozen record → flattened leaves
Custom type	register_leaf → your SSA value

Records flatten into their fields

Dialect-neutral record

```
@m.struct
class Vec2:
    x: m.Float32
    y: m.Float32

@m.jit
def norm2(v: Vec2):
    return v.x*v.x + v.y*v.y
```

Two fields → two SSA arguments

```
func.func @norm2(
    %x: f32, %y: f32) → f32 {
    %a = arith.mulf %x, %x : f32
    %b = arith.mulf %y, %y : f32
    %r = arith.addf %a, %b : f32
    func.return %r : f32
}

// norm2(Vec2(x=3, y=4)) → 25
```

Frozen record · no aggregate SSA type

Extend values with mydialect.Tensor

Custom leaf · implementation excerpt

```
@dataclass(frozen=True)
class Tensor:
    value: ir.Value

    register_leaf(
        Tensor,
        ir_types=lambda t: [t.value.type],
        ir_values=lambda t: [t.value],
        from_ir_values=lambda t, vs: Tensor(vs[0]),
    )
```

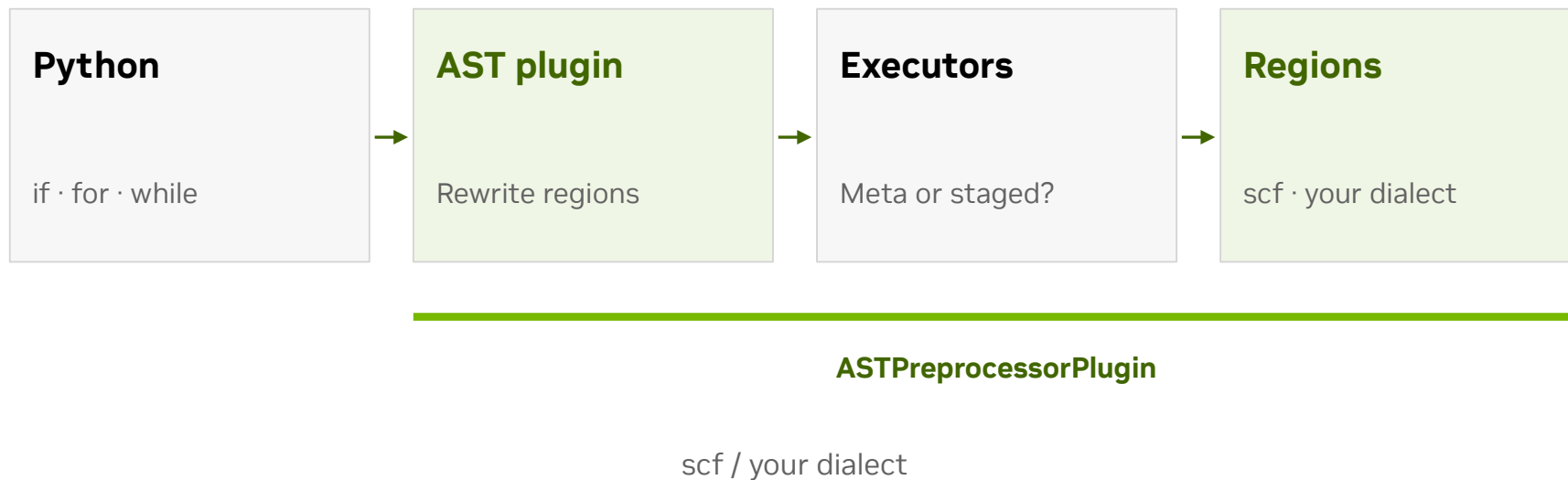
Keep a custom value

Own the Python wrapper

Cross DSL boundaries

Extract + rebuild SSA leaves

5 · Native control flow



Syntax capture and IR emission are separate choices.

Assigned name → iter_arg

Python · find the carry

```
acc = m.Int32(0)
for i in range(n):
    acc = acc + i

# Defined outside: acc
# Written inside:  acc
# write_args_names = ["acc"]
```

MLIR · thread the SSA value

```
%sum = scf.for %i = %c0 to %n
      step %c1 iter_args(%acc = %zero)
      → (i32) : i32 {
    %next = arith.addi %acc, %i : i32
    scf.yield %next : i32
  }
```

Flatten → build region → yield → rebuild

Capture has an explicit boundary

Python / Meta	Staged
if / for / while	scf.if / scf.for / scf.while
Known early exit	Python executes it
Data-dependent early exit	UNSUP_EARLY_EXIT
Frozen dataclass	Flatten + rebuild leaves

Replace capture through one role

AST plugin · implementation excerpt

```
class MyAST(scf.ASTPreprocessor):  
    def executors(self, dsl):  
        hooks = super().executors(dsl)  
        hooks["loop_execute_range_dynamic"] = (  
            my_loop_executor)  
        return hooks  
  
# In the Plugins record:  
ast_preprocessor=MyAST()
```

Reuse the rewrite

Replace a region builder

Or replace both

preprocessor_class + executors

6 · Explicit builders

Python · implementation excerpt

```
@m.jit(preprocess=False)
def sum_squares(n: m.Int32):
    for i, acc, result in m.for_(
        0, n, 1, [m.Int32(0)]):
        m.yield_(acc + i * i)
    return result
```

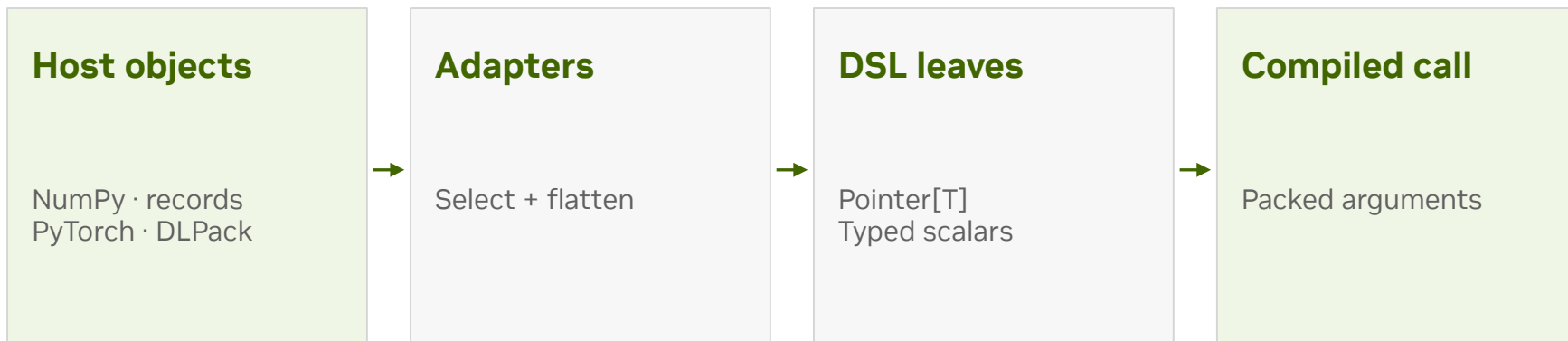
No syntax rewrite

Build regions directly

Same building blocks

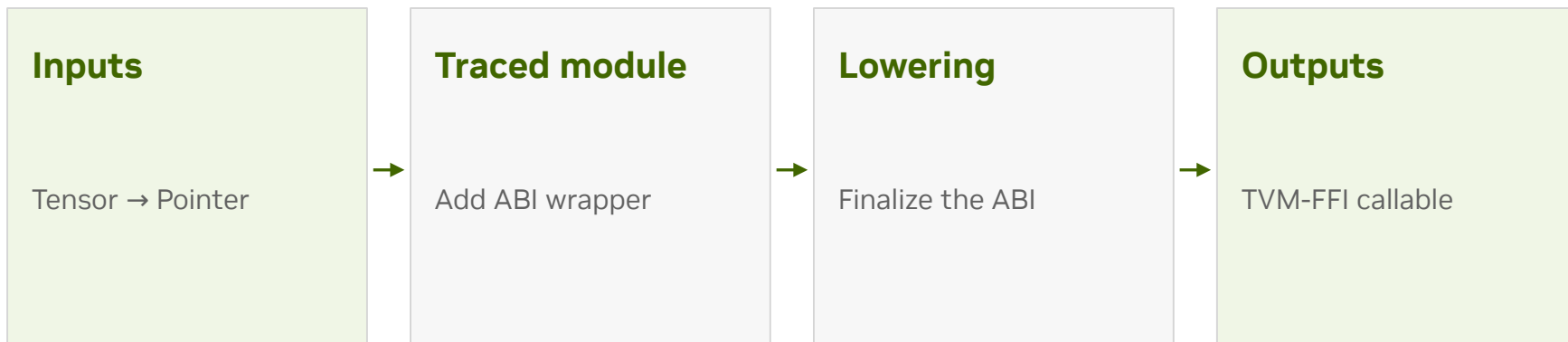
for_ · if_ · while_ · yield_

7 · Bring host values to the DSL



Pointer ABI: contiguous buffers · dimensions passed separately

Adapters work in both directions



AdapterPlugin · register / attach / after_lowering / wrap

8 · @kernel comes from a decorator plugin

GPU plugin + kernel · excerpt

```
plugins = Plugins(  
    # ... selected roles ...  
    decorators=[gpu.Kernels(  
        chip_option="cubin-chip")],  
)  
  
@m.kernel  
def add_one(p: m.Pointer[m.Float32]):  
    i = m.thread_idx()[0]  
    p[i] = p[i] + 1.0
```

Core decorator

@jit

Decorator family

Adds @kernel + launcher

Launch is an explicit operation

Python · implementation excerpt

```
@m.jit
def run(p: m.Pointer[m.Float32]):
    add_one(p).launch(
        m.LaunchConfig(
            grid=[1], block=[32]))

# Caller owns a 32-element CUDA buffer.
run(device_tensor)
```

Prepare → launch

gpu.func → gpu.launch_func

Guard + validate

Grid size · buffer kind

9 · The DSL owns the pass pipeline

Owner	Responsibility
Sub-DSL	pipeline() · register_dialects(ctx)
CompilerPlugin	compile · jit · load
FuncEntryPlugin	Entry op · results ABI
ExecutionEngine compiler	LLVM lowering + packed ctypes call

Compile now. Call later.

Python · implementation excerpt

```
@m.jit
def affine(x: m.Int32, scale, bias):
    return x * scale + bias

compiled = m.compile(affine, 7, 3, 4)
result = compiled(7, 3, 4)
int(result) # 25
```

m.compile

Specialize + compile

Compiled callable

Invoke later

10 · Cache compilation work

Cache	What is reused
In memory	Traced-module key → compiled callable
On disk	Lowered bytecode + CRC32
Configuration	CACHE_DIR · DISABLE_FILE_CACHING
Observation	cache_hits · file_cache_hits

11 · Generate IR and stop

IR-only DSL · setup excerpt

```
class RecipeDSL(BaseDSL):
    plugins = Plugins(
        type_ops=TypeOps(scalars=arith),
        func_entry=func.Entry(),
        compiler=None,
    )

# Or use an existing DSL:
# MLIR_DSL_DRYRUN=1 MLIR_DSL_PRINT_IR=1
```

Compiler is optional

Trace result only

Useful for

Tests · recipe consumers

12 · Errors should point to Python

Python · implementation excerpt

```
@m.jit
def bad(x: m.Int32):
    return int(x)

# PHASE_REQUIRES_CONSTANT
# Runtime value used as Python value
```

Stable codes

Catalogue + suggestions

Useful context

Source frame + DSL message

13 · Locations and remarks

Mechanism	What you see
DEBUGINFO	Python traceback → MLIR locations
REMARKS	Structured trace + compiler feedback
Source frames	Diagnostics point back to the function
Compiler role	Owns the remark session

14 · See what the compiler does

<PREFIX>_*	Purpose
PRINT_IR · DRYRUN	Inspect the traced program
PROFILE_COMPILER	Time compilation phases
JIT_TIME_PROFILING	Measure JIT work
LOG_LEVEL	Control logging

15 · One subclass. One Plugins record.

Downstream DSL · implementation excerpt

```
class MyDSL(BaseDSL):  
    plugins = Plugins(  
        type_ops=TypeOps(  
            scalars=arith, vectors=vector, memory=llvm),  
        func_entry=func.Entry(),  
        ast_preprocessor=scf.ASTPreprocessor(),  
        compiler=execution_engine.Compiler(),  
        decorators=[gpu.Kernels()],  
        adapters=[pytorch.PyTorchPlugin()],  
    )
```

Select roles

One implementation per role

Extend families

Any number of plugins

Own the namespace and lowering

Downstream DSL · methods

```
def __init__(self):
    super().__init__(
        name="MY_DSL",
        dsl_package_name=["mydsl"],
        preprocess=True)

def register_dialects(self, ctx):
    mydialect.register_dialect(ctx)
```

Downstream DSL · pipeline excerpt

```
def pipeline(self):
    return [
        "convert-mydialect-to-llvm",
        "convert-scf-to-cf",
        "convert-cf-to-llvm",
        "convert-arith-to-llvm",
        "convert-func-to-llvm",
        "reconcile-unrealized-casts",
    ]
```

Reuse a composition; change one field

Python · implementation excerpt

```
from dataclasses import replace

class CpuOnly(m.MlirTestDSL):
    plugins = replace(
        m.MlirTestDSL.plugins,
        decorators=(),
        adapters=(),
    )
```

Keep the roles

Types · entry · AST · compiler

Change the families

Drop GPU + optional adapters

Every plugin has a small shared lifecycle

Shared hook	Meaning
available()	Probe optional support
install(dsl)	Bind this instance
register_dialects(ctx)	Register owned dialects
shared_libs()	Provide runtime libraries

One program: Python recipe + staged loop

Python · executed

```
@m.jit
def fused(arr: m.Pointer[m.Float32],
         ops):
    for i in m.range(4):
        x = arr[i]
        for op in ops.values():
            x = op.apply(x)
        arr[i] = x

# [0, 1, 2, 3] → [6, 9, 12, 15]
```

MLIR · loop body

```
%p = llvm.getelementptr %arr[%i]
      : (!llvm.ptr, i32) → !llvm.ptr, f32
%x = llvm.load %p : !llvm.ptr → f32
%a = arith.addf %x, %two : f32
%r = arith.mulf %a, %three : f32
llvm.store %r, %p : f32, !llvm.ptr
scf.yield
```

Staged array loop · Meta recipe loop

From loop carries to richer mutation

Aliasing · mutable state · PyIR

— Amir's section

Advanced mutation tracking

Amir supplies this section.

——— Return to Guray for a target example.

Tile IR: choose target implementations

Downstream port sketch · proposed API

```
class TileDSL(BaseDSL):  
    plugins = Plugins(  
        type_ops=tile.TypeOps(),  
        func_entry=tile.Entry(),  
        ast_preprocessor=tile.ASTPreprocessor(),  
        compiler=tile.Compiler(),  
        decorators=[tile.Kernels()],  
        adapters=[dlpack.DlpackPlugin()],  
    )
```

Replace target roles

Types · entry · capture · compiler

Reuse infrastructure

Cache · diagnostics · adapters

Tile IR: one type-ops role

Python value	Proposed Tile representation
Int32 / Float32	Rank-0 tile
Vector	Rank-1 tile
Pointer[T]	Tile of pointers
+ · load · store	Target arithmetic + memory ops

Tile IR: choose its control-flow model

Downstream port sketch · proposed API

```
class TileAST(ASTPreprocessorPlugin):  
    preprocessor_class = TilePreprocessor  
  
    def executors(self, dsl):  
        return tile_region_builders(dsl)  
  
# In the Plugins record:  
ast_preprocessor=TileAST()
```

Reuse capture where it fits

Python syntax → target regions

Own target semantics

Early exits · carried values

Tile IR: keep a small user surface

Downstream port sketch · user API

```
@tile.kernel
def add(a: tile.Tensor,
        b: tile.Tensor,
        out: tile.Tensor):
    x = tile.load(a)
    y = tile.load(b)
    tile.store(out, x + y)
```

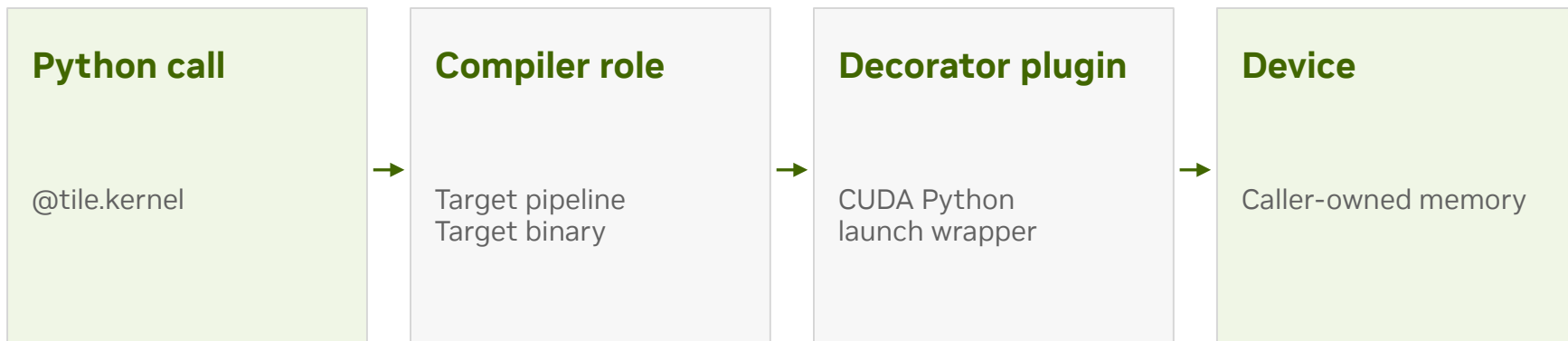
Proposed plugin ownership

```
tile.TypeOps
    types + operations

tile.Kernels
    entry + launch

tile.Compiler
    binary + runtime
```

Tile IR: the target owns launch



Port sketch · reusable core, target-owned execution